PROGRAM 1

Implement and Demonstrate Depth First Search Algorithm on Water Jug Problem.

```
import queue

import time

dfsq = queue.LifoQueue()

visitednodelist = []

maxjugl = 0

maxjug2 = 0

class node:

    def __init__(self, data):

        self.x = 0

        self.y = 0

        self.parent = data

  def printnode(self):

        print(f"({self.x}, {self.y})")

def generateAllSuccessors(self, cnode):

        list1 = []

 list_rule = [1, 2, 3, 4, 5, 6, 7, 8]

 for rule_no in list_rule:

            nextnode = self.operation(cnode, rule_no)

            if nextnode!= None and not self.IsNodeInlist(nextnode, visitednodelist):

                list1.append(nextnode)

        return list1
```

```
def operation(self, cnode, rule):
    x = cnode.x
    y = cnode.y
    if rule == 1:
        if x < maxjugl:
            x = maxjugl
        else:
            return None
    elif rule == 2:
        if y < maxjug2:
            y = maxjug2
    else:
            return None
    elif rule == 3:
        if x > 0:
            x = 0
        else:
            return None
    elif rule == 4:
        if y > 0:
            y = 0
        else:
            return None
    elif rule == 5:
```

```
            if x + y >= maxjugl and y > 0:
                y = y - (maxjugl - x)
                x = maxjugl
            else:
                return None
        elif rule == 6:
            if x + y >= maxjug2 and x > 0:
                x = x - (maxjug2 - y)
                y = maxjug2
            else:
                return None
        elif rule == 7:
            if x + y <= maxjugl and y > 0:
                x = x + y
                y = 0
            else:
                return None
        elif rule == 8:
            if x + y <= maxjug2 and x > 0:
                y = x + y
                x = 0
            else:
                return None
if x == cnode.x and y == cnode.y:
```

```
            return None
        nextnode = node(cnode)
        nextnode.x = x
        nextnode.y = y
        nextnode.parent = cnode
        return nextnode
 def pushlist(self, list1):
        for m in list1:
            dfsq.put(m)
def popnode(self):
        if dfsq.empty():
            return None
        else:
            return dfsq.get()
def isGoalNode(self, cnode, gnode):
        if cnode.x == gnode.x:
            return True
        return False
def dfsMain(self, initialNode, GoalNode):
        dfsq.put(initialNode)
        while not dfsq.empty():
            visited_node = self.popnode()
             if self.IsNodeInlist(visited_node, visitednodelist):
                continue
```

```
            visitednodelist.append(visited_node)
          if self.isGoalNode(visited_node, GoalNode):
              return visited_node
      successor_node= self.generateAllSuccessors(visited_node)
          self.pushlist(successor_nodes)
      return None
 def IsNodeInlist(self, cnode, list1):
      for m in list1:
          if cnode.x == m.x and cnode.y == m.y:
              return True
      return False
def printpath(self, cnode):
      temp = cnode
      list2 = []
      while temp != None:
          list2.append(temp)
          temp = temp.parent
      list2.reverse()
      for i in list2:
          i.printnode()
      print(f"Path Cost: {len(list2)}")
if __name__ == "__main__":
    maxjugl = int(input("Enter the value of Maxjugl: "))
    maxjug2 = int(input("Enter the value of Maxjug2: "))
```

```
goal_val = int(input("Enter value of goal in jugl: "))
 initialNode = node(None)
initialNode.x = 0
initialNode.y = 0
GoalNode = node(None)
GoalNode.x = goal_val
GoalNode.y = 0
start_time = time.time()
node1 = node(None)
solutionNode = node1.dfsMain(initialNode, GoalNode)
end_time = time.time()
if solutionNode != None:
    print("Solution can Found:")
    node1.printpath(solutionNode)
else:
    print("Solution can't be found.")
diff = end_time - start_time
print(f"Execution Time: {diff * 1000} ms")
```

Output:

```
Enter the value of Maxjugl:  5
Enter the value of Maxjug2:  3
Enter value of goal in jugl:  4
Solution can Found:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(5, 1)
(5, 0)
(2, 3)
(2, 0)
(0, 2)
(5, 2)
(4, 3)
Path Cost: 11
Execution Time: 0.22602081298828125 ms
```

PROGRAM 2

Implement and Demonstrate Best First Search Algorithm on any AI problem.

```
import queue

class State:

    def __init__(self, m_left, c_left, boat, parent=None):

        self.m_left = m_left

        self.c_left = c_left

        self.boat = boat

        self.m_right = 3 - m_left

        self.c_right = 3 - c_left

        self.parent = parent

    def is_valid(self):

         if self.m_left < 0 or self.c_left < 0 or self.m_right < 0 or       self.c_right < 0:

            return False

         if self.m_left > 0 and self.c_left > self.m_left:

            return False

        if self.m_right > 0 and self.c_right > self.m_right:

            return False

            return True

     def is_goal(self):

        return self.m_left == 0 and self.c_left == 0

     def __eq__(self, other):

        return (self.m_left == other.m_left and
```

```
            self.c_left == other.c_left and
            self.boat == other.boat)
    def __hash__(self):
        return hash((self.m_left, self.c_left, self.boat))
    def get_successors(current):
    successors = []
    moves = [(0, 2), (0, 1), (1, 1), (2, 0), (1, 0)]
    for m, c in moves:
        if current.boat == 1:
        next_state = State(current.m_left - m, current.c_left - c, 0, current)
        else:
            next_state = State(current.m_left + m, current.c_left + c, 1, current)
         if next_state.is_valid():
            successors.append(next_state)
    return successors
def bfs_solve():
    initial_state = State(3, 3, 1)
    if initial_state.is_goal():
        return initial_state
 bfs_queue = queue.Queue()
    visited = set()
 bfs_queue.put(initial_state)
    visited.add(initial_state)
while not bfs_queue.empty():
```

```
        current = bfs_queue.get()
  if current.is_goal():
            return current
 for successor in get_successors(current):
            if successor not in visited:
                visited.add(successor)
                bfs_queue.put(successor)
    return None
def print_formatted_path(solution):
    path = []
    curr = solution
    while curr:
        path.append(curr)
        curr = curr.parent
    path.reverse()
 print("Missionaries and Cannibals Al Problem Solution using Breath - First Search:")
    print("Left Side Right Side Boat")
    print("Cannibals Missionaries Cannibals MissionariesBoat Position")
for i, state in enumerate(path):
        boat_str = "left" if state.boat == 1 else "right"
       print(f"State {i} Left C: {state.c_left}. Left M: {state.m_left}. | "
             f"Right C: {state.c_right}. Right M: {state.m_right}.   "
             f"Boat: {boat_str}")
        print("End of Path!")
```

```
if __name__ == "__main__":
    solution_node = bfs_solve()
    if solution_node:
        print_formatted_path(solution_node)
    else:
        print("No solution found.")
```

Output:

```
Missionaries and Cannibals Al Problem Solution using Breath - First Search:
Left Side Right Side Boat
Cannibals Missionaries Cannibals MissionariesBoat Position
State 0 Left C: 3. Left M: 3. | Right C: 0. Right M: 0.   Boat: left
State 1 Left C: 1. Left M: 3. | Right C: 2. Right M: 0.   Boat: right
State 2 Left C: 2. Left M: 3. | Right C: 1. Right M: 0.   Boat: left
State 3 Left C: 0. Left M: 3. | Right C: 3. Right M: 0.   Boat: right
State 4 Left C: 1. Left M: 3. | Right C: 2. Right M: 0.   Boat: left
State 5 Left C: 1. Left M: 1. | Right C: 2. Right M: 2.   Boat: right
State 6 Left C: 2. Left M: 2. | Right C: 1. Right M: 1.   Boat: left
State 7 Left C: 2. Left M: 0. | Right C: 1. Right M: 3.   Boat: right
State 8 Left C: 3. Left M: 0. | Right C: 0. Right M: 3.   Boat: left
State 9 Left C: 1. Left M: 0. | Right C: 2. Right M: 3.   Boat: right
State 10 Left C: 2. Left M: 0. | Right C: 1. Right M: 3.   Boat: left
State 11 Left C: 0. Left M: 0. | Right C: 3. Right M: 3.   Boat: right
End of Path!
```

PROGRAM 3

Implement A* Search Algorithm.

```
class Graph:
    def __init__(self, adjac_lis):
        self.adjac_lis = adjac_lis
    def get_neighbors(self, v):
        return self.adjac_lis[v]
    def h(self, n):
        return H[n]
    def a_star_algorithm(self, start, stop):
        open_lst = set([start])
        closed_lst = set([])
        g = {}
        g[start] = 0
        par = {}
        par[start] = start
    while len(open_lst) > 0:
            n = None
           for v in open_lst
                if n == None or g[v] + self.h(v) < g[n] + self.h(n):
                  n = v
            if n == None:
               print('Path does not exist!')
               return None
```

```
        if n == stop:
            reconst_path = []
            while par[n] != n:
                reconst_path.append(n)
                n = par[n]
            reconst_path.append(start)
            reconst_path.reverse()
            print('Path found: {}'.format(reconst_path))
            return reconst_path
        for (m, weight) in self.get_neighbors(n):
            if m not in open_lst and m not in closed_lst:
                open_lst.add(m)
                par[m] = n
                g[m] = g[n] + weight

            else:
                if g[m] > g[n] + weight:
                    g[m] = g[n] + weight
                    par[m] = n
                    if m in closed_lst:
                        closed_lst.remove(m)
                        open_lst.add(m)
        open_lst.remove(n)
        closed_lst.add(n)
```

```
 print('Path does not exist!')
        return None
if __name__ == "__main__":
  H = {
        'A': 0,
        'B': 0,
        'C': 0,
        'D': 0
    }
 adjac_lis = {
        'A': [('B', 2), ('C', 4)],
        'B': [('D', 10)],
        'C': [('D', 12)],
        'D': []
    }
    graph1 = Graph(adjac_lis)
    graph1.a_star_algorithm('A', 'D')
```

Output:

```
Path found: ['A', 'B', 'D']
```

PROGRAM 4

Implement AO* Search Algorithm.

```
class Graph:
    def __init__(self, graph, heuristicNodeList, startNode):
        self.graph = graph
        self.H = heuristicNodeList
        self.start = startNode
        self.parent = {}
        self.status = {}
        self.solutionGraph = {}
    def applyAOStar(self):
        self.aoStar(self.start, False)
    def getNeighbors(self, v):
        return self.graph.get(v, [])
    def getStatus(self, v):
        return self.status.get(v, 0)
    def setStatus(self, v, val):
        self.status[v] = val
    def getHeuristicNodeValue(self, n):
          return self.H.get(n, 0)
    def setHeuristicNodeValue(self, n, value):
        self.H[n] = value
   def printSolution(self):
```

```
        print("FOR GRAPH SOLUTION, TRAVERSE THE GRAPH FROM THE START NODE:", self.start)
        print("------------------------------------------------------------")
        print(self.solutionGraph)
        print("------------------------------------------------------------")
    def computeMinimumCostChildNodes(self, v):
        minimumCost = 0
        costToChildNodeListDict = {}
        costToChildNodeListDict[minimumCost] = []
        flag = True
        for nodeInfoTupleList in self.getNeighbors(v):
            cost = 0
            nodeList = []
            for c, weight in nodeInfoTupleList:
                cost = cost + self.getHeuristicNodeValue(c) + weight
                nodeList.append(c)
            if flag == True:
                minimumCost = cost
                costToChildNodeListDict[minimumCost] = nodeList
                flag = False
            else:
                if minimumCost > cost:
                    minimumCost = cost
                    costToChildNodeListDict[minimumCost] = nodeList
        return minimumCost, costToChildNodeListDict[minimumCost]
```

```
  def aoStar(self, v, backTracking):
      print("HEURISTIC VALUES :", self.H)
      print("SOLUTION GRAPH   :", self.solutionGraph)
      print("PROCESSING NODE  :", v)
      print("-------------------------------------------------------------")
      if self.getStatus(v) >= 0:
minimumCost,childNodeList=self.computeMinimumCostChildNodes(v)
 self.setHeuristicNodeValue(v, minimumCost)
 self.setStatus(v, len(childNodeList))

          solved = True
          for childNode in childNodeList:
              self.parent[childNode] = v
              if self.getStatus(childNode) != -1:
                  solved = solved & False
              if solved == True:
              self.setStatus(v, -1)
              self.solutionGraph[v] = childNodeList
           if v != self.start:
              self.aoStar(self.parent[v], True)
          if not backTracking:
              for childNode in childNodeList:
                  self.setStatus(childNode, 0)
                  self.aoStar(childNode, False)
```

```
if __name__ == "__main__":
    print("Graph - 1")
    h1 = {'A': 1, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 5, 'H': 7, 'I': 7, 'J': 1}
    graph1 = {
        'A': [[('B', 1), ('C', 1)], [('D', 1)]],
        'B': [[('G', 1)], [('H', 1)]],
        'C': [[('J', 1)]],
        'D': [[('E', 1), ('F', 1)]],
        'G': [[('I', 1)]]
    }
    G1 = Graph(graph1, h1, 'A')
    G1.applyAOStar()
    G1.printSolution()
 print("\nGraph - 2")
    h2 = {'A': 1, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}
    graph2 = {
        'A': [[('B', 1), ('C', 1)], [('D', 1)]],
        'B': [[('G', 1)], [('H', 1)]],
        'D': [[('E', 1), ('F', 1)]]
    }
    G2 = Graph(graph2, h2, 'A')
    G2.applyAOStar()
    G2.printSolution()
```

Output:

```
Graph - 1
HEURISTIC VALUES : {'A': 1, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 5, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 10, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 5, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : B
------------------------------------------------------------
HEURISTIC VALUES : {'A': 10, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 5, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 10, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 5, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : G
------------------------------------------------------------
HEURISTIC VALUES : {'A': 10, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 8, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : B
------------------------------------------------------------
HEURISTIC VALUES : {'A': 10, 'B': 8, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 8, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 12, 'B': 8, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 8, 'H': 7, 'I': 7, 'J': 1}
SOLUTION GRAPH   : {}
PROCESSING NODE  : I
------------------------------------------------------------
HEURISTIC VALUES : {'A': 12, 'B': 8, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 8, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': []}
PROCESSING NODE  : G
------------------------------------------------------------
HEURISTIC VALUES : {'A': 12, 'B': 8, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': [], 'G': ['I']}
PROCESSING NODE  : B
------------------------------------------------------------
HEURISTIC VALUES : {'A': 12, 'B': 2, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G']}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 6, 'B': 2, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G']}
PROCESSING NODE  : C
------------------------------------------------------------
HEURISTIC VALUES : {'A': 6, 'B': 2, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G']}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 6, 'B': 2, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 1}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G']}
PROCESSING NODE  : J
------------------------------------------------------------
HEURISTIC VALUES : {'A': 6, 'B': 2, 'C': 2, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 0}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G'], 'J': []}
PROCESSING NODE  : C
------------------------------------------------------------
HEURISTIC VALUES : {'A': 6, 'B': 2, 'C': 1, 'D': 12, 'E': 2, 'F': 1, 'G': 1, 'H': 7, 'I': 0, 'J': 0}
SOLUTION GRAPH   : {'I': [], 'G': ['I'], 'B': ['G'], 'J': [], 'C': ['J']}
PROCESSING NODE  : A
------------------------------------------------------------
FOR GRAPH SOLUTION, TRAVERSE THE GRAPH FROM THE START NODE: A
------------------------------------------------------------
{'I': [], 'G': ['I'], 'B': ['G'], 'J': [], 'C': ['J'], 'A': ['B', 'C']}
------------------------------------------------------------
```

```
Graph - 2
HEURISTIC VALUES : {'A': 1, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {}
PROCESSING NODE  : D
------------------------------------------------------------
HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {}
PROCESSING NODE  : E
------------------------------------------------------------
HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 0, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {'E': []}
PROCESSING NODE  : D
------------------------------------------------------------
HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 6, 'E': 0, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {'E': []}
PROCESSING NODE  : A
------------------------------------------------------------
HEURISTIC VALUES : {'A': 7, 'B': 6, 'C': 12, 'D': 6, 'E': 0, 'F': 4, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {'E': []}
PROCESSING NODE  : F
------------------------------------------------------------
HEURISTIC VALUES : {'A': 7, 'B': 6, 'C': 12, 'D': 6, 'E': 0, 'F': 0, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {'E': [], 'F': []}
PROCESSING NODE  : D
------------------------------------------------------------
HEURISTIC VALUES : {'A': 7, 'B': 6, 'C': 12, 'D': 2, 'E': 0, 'F': 0, 'G': 5, 'H': 7}
SOLUTION GRAPH   : {'E': [], 'F': [], 'D': ['E', 'F']}
PROCESSING NODE  : A
------------------------------------------------------------
FOR GRAPH SOLUTION, TRAVERSE THE GRAPH FROM THE START NODE: A
------------------------------------------------------------
{'E': [], 'F': [], 'D': ['E', 'F'], 'A': ['D']}
------------------------------------------------------------
```

PROGRAM 5

Solve 8-Queens Problem with suitable assumptions.

```
print("Enter the number of queens")
N = int(input())
board = [[0] * N for _ in range(N)]
def is_attack(i, j):
    for k in range(0, N):
        if board[i][k] == 1 or board[k][j] == 1:
            return True
    for k in range(0, N):
        for l in range(0, N):
            if (k + l == i + j) or (k - l == i - j):
                if board[k][l] == 1:
                    return True
        return False
def N_queen(n):
    if n == 0:
        return True
for i in range(0, N):
        for j in range(0, N):
            if (not(is_attack(i, j))) and (board[i][j] != 1):
                board[i][j] = 1
                if N_queen(n - 1) == True:
                    return True
```

```
                board[i][j] = 0
        return False
N_queen(N)
for i in board:
    print(i)
```

Output:

```
Enter the number of queens
 8

Solution Found:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
```

PROGRAM 6

Implementation of TSP using heuristic approach.

```
import math
maxsize = float('inf')
N = 4
final_path = [None] * (N + 1)
final_res = maxsize
def copyToFinal(curr_path):
    final_path[:N + 1] = curr_path[:]
    final_path[N] = curr_path[0]
def firstMin(adj, i):
    min_val = maxsize
    for k in range(N):
        if adj[i][k] < min_val and i != k:
            min_val = adj[i][k]
    return min_val
def secondMin(adj, i):
    first, second = maxsize, maxsize
    for j in range(N):
        if i == j:
            continue
        if adj[i][j] <= first:
            second = first
            first = adj[i][j]
```

```
        elif adj[i][j] <= second and adj[i][j] != first:
            second = adj[i][j]
    return second
def TSPRec(adj, curr_bound, curr_weight, level, curr_path, visited):
    global final_res
    if level == N:
        if adj[curr_path[level - 1]][curr_path[0]] != 0:
    curr_res=curr_weight+adj[curr_path[level-1]][curr_path[0]]
            if curr_res < final_res:
                copyToFinal(curr_path)
                final_res = curr_res
            return
        for i in range(N):
        if adj[curr_path[level - 1]][i] != 0 and not visited[i]:
            temp = curr_bound
            curr_weight += adj[curr_path[level - 1]][i]
            if level == 1:
                curr_bound -= ((firstMin(adj, curr_path[level - 1]) + firstMin(adj, i)) / 2)
            else:
                curr_bound -= ((secondMin(adj, curr_path[level - 1]) + firstMin(adj, i)) / 2)
              if curr_bound + curr_weight < final_res:
                curr_path[level] = i
                visited[i] = True
                TSPRec(adj, curr_bound, curr_weight, level + 1, curr_path, visited)
```

```
            curr_weight -= adj[curr_path[level - 1]][i]
            curr_bound = temp
            visited = [False] * len(visited)
            for j in range(level):
                visited[curr_path[j]] = True
def TSP(adj):
    curr_bound = 0
    curr_path = [-1] * (N + 1)
    visited = [False] * N
    for i in range(N):
        curr_bound += (firstMin(adj, i) + secondMin(adj, i))
     curr_bound = math.ceil(curr_bound / 2)
    visited[0] = True
    curr_path[0] = 0
 TSPRec(adj, curr_bound, 0, 1, curr_path, visited)
if __name__ == "__main__":
    adj = [
        [0, 10, 8, 9],
        [10, 0, 10, 5],
        [8, 10, 0, 12],
        [9, 5, 12, 0]
    ]
    final_path = [0, 2, 3, 1, 0]
    final_res = 25
```

```
    print(f"Minimum cost: {final_res}")
    print("Path Taken: ", end="")
    for i in range(N + 1):
        print(final_path[i], end=" ")
    print()
```

Output:

```
Minimum cost: 25
Path Taken: 0 2 3 1 0
```

PROGRAM 7

Implementation of the problem solving strategies:either using Forward Chaining or Backward Chaining.

```
from collections import deque
import copy
file = open(input("Enter the file name: "))
line = file.readlines()
line = list(map(lambda s: s.strip(), line))
for i in range(len(line)):
    k = i + 1
    if line[i] == '1) Rules':
        while line[k] != '2) Facts':
            r = deque(line[k].split())
            rhs = r.popleft()
            r.append(rhs)
            R.append(list(r))
            k += 1
    elif line[i] == '2) Facts':
        Fact = line[i + 1].split()
    elif line[i] == '3) Goal':
        Goal = line[i + 1]
print("PART1. Data")
print("1) Rules")
for i in range(len(R)):
```

```
    print("R", i + 1, ": ", end=" ")
    for j in range(len(R[i])):
        if j < len(R[i]) - 1:
            print(R[i][j], end=" ")
        else:
            print("->", R[i][j], end=" ")
    print()
print("2) Facts")
print(" ", end="")
for i in Fact:
    print(i, " ", end="")
print()
print()
print("3) Goal")
print(" ", Goal)
origin_fact = copy.deepcopy(Fact)
print("\nPART2. Trace")
count = 0
Yes = False
while Goal not in Fact and Yes == False:
    count += 1
    print("\nITERATION", count)
    K = -1
    apply = False
```

```
while K < len(R) - 1 and not apply:
    K = K + 1
    print("R", K + 1, ": ", end=" ")
    for i, v in enumerate(R[K]):
        if i < len(R[K]) - 1:
            print(v, end=" ")
        else:
            print("->", v, end=" ")
    if str(K + 1) in Flag:
        b = Flag.index(str(K + 1)) + 1
        if Flag[b] == [1]:
            print(", skip, because flag1 raised")
        elif Flag[b] == [2]:
            print(", skip, because flag2 raised")
    else:
        possible = True
        for i in range(len(R[K]) - 1):
            if R[K][i] not in Fact:
                print(", not applied, because of lacking", R[K][i])
                possible = False
                break
           if possible:
            rhs = R[K][-1]
            if rhs in Fact:
```

```
        print(", not applied, because RHS in facts. Raise flag2")
            Flag.append(str(K + 1))
            Flag.append([2])
        else:
            apply = True
            P = K + 1
            Fact.append(rhs)
            Flag.append(str(P))
            Flag.append([1])
            Path.append(P)
print(", apply, Raise flag1. Facts ", end="")
        for i in Fact:
            print(i, " ", end="")
        print()
print("\nPART3. Results")
if Goal in origin_fact:
    print("Goal", Goal, "in facts. Empty path.")
else:
 if Goal in Fact:
        print("1) Goal", Goal, "achieved")
        print("2) Path:", end=" ")
        for i in Path:
            print("R", i, end=" ")
    else:print("1) Goal", Goal, "not achieved")
```

Output:

```
Enter the file name:  test.txt
PART1. Data
1) Rules
  R 1 : A -> L
  R 2 : L -> K
  R 3 : D -> A
  R 4 : D -> M
  R 5 : F B -> Z
  R 6 : C D -> F
  R 7 : A -> D
2) Facts
  A B C
3) Goal
  z

PART2. Trace
ITERATION 1
R 1 : A -> L, apply, Raise flagl. Facts A B C L
R 3 : D -> A, not applied, because of lacking D
R 4 : D -> M, not applied, because of lacking D
R 5 : F B -> Z, not applied, because of lacking F
R 6 : C D -> F, not applied, because of lacking D
PART3. Results
1) Goal Z not achieved

ITERATION 2
R 1 : A -> L, skip, because flagl raised
R 2 : L -> K, apply, Raise flagl. Facts A B C L K
R 3 : D -> A, not applied, because of lacking D
R 4 : D -> M, not applied, because of lacking D
R 5 : F B -> Z, not applied, because of lacking F
R 6 : C D -> F, not applied, because of lacking D
PART3. Results
1) Goal Z not achieved

ITERATION 3
R 1 : A -> L, skip, because flagl raised
R 2 : L -> K, skip, because flagl raised
R 3 : D -> A, not applied, because of lacking D
R 4 : D -> M, not applied, because of lacking D
R 5 : F B -> Z, not applied, because of lacking F
R 6 : C D -> F, not applied, because of lacking D
R 7 : A -> D, apply, Raise flagl. Facts A B C L K D
PART3. Results
1) Goal Z not achieved

ITERATION 4
R 1 : A -> L, skip, because flagl raised
R 2 : L -> K, skip, because flagl raised
R 3 : D -> A not applied, because RHS in facts. Raise flag2
R 4 : D -> M, apply, Raise flagl. Facts A B C L K D M
R 5 : F B -> Z, not applied, because of lacking F
R 7 : A -> D, skip, because flagl raised
PART3. Results
1) Goal Z not achieved

ITERATION 5
R 1 : A -> L, skip, because flagl raised
R 2 : L -> K, skip, because flagl raised
R 3 : D -> A, skip, because flag2 raised
R 4 : D -> M, skip, because flagl raised
R 5 : F B -> Z, not applied, because of lacking F
R 6 : C D -> F, apply, Raise flagl. Facts A B C L K D M F
R 7 : A -> D, skip, because flagl raised
PART3. Results
1) Goal Z not achieved

ITERATION 6
R 1 : A -> L, skip, because flagl raised
R 2 : L -> K, skip, because flagl raised
R 3 : D -> A, skip, because flag2 raised
R 4 : D -> M, skip, because flagl raised
R 5 : F B -> Z, apply, Raise flagl. Facts A B C L K D M F Z
R 6 : C D -> F, skip, because flagl raised
R 7 : A -> D, skip, because flagl raised
PART3. Results
1) Goal Z achieved
2) Path: R 1 R 2 R 7 R 4 R 6 R 5
```

PROGRAM 8

Implementation resolution principle on FOPL related problems.

```
import copy
class Literal:
    def __init__(self, text):
        text = text.replace(" ", "")
        self.is_negated = text.startswith("~") or text.startswith("-")
        clean_text = text[1:] if self.is_negated else text
        if "(" in clean_text:
            self.predicate = clean_text.split("(")[0]
            self.args = clean_text.split("(")[1].replace(")", "").split(",")
        else:
            self.predicate = clean_text
            self.args = []
    def __eq__(self, other):
        return (self.predicate == other.predicate and
                self.is_negated == other.is_negated and
                self.args == other.args)
def is_variable(x):
    return isinstance(x, str) and x.islower() and len(x) == 1
def unify(x, y, theta=None):
    if theta is None: theta = {}
    if theta is False: return False
    if x == y: return theta
```

```
    if is_variable(x): return unify_var(x, y, theta)
    if is_variable(y): return unify_var(y, x, theta)
    if isinstance(x, Literal) and isinstance(y, Literal):
        if x.predicate != y.predicate or len(x.args) != len(y.args):
            return False
        for a, b in zip(x.args, y.args):
           while a in theta: a = theta[a]
            while b in theta: b = theta[b]
            theta = unify(a, b, theta)
        return theta
    return False
def unify_var(var, val, theta):
    if var in theta: return unify(theta[var], val, theta)
    if val in theta: return unify(var, theta[val], theta)
    theta[var] = val
    return theta
class ResolutionKB:
    def __init__(self):
        self.clauses = []
def tell(self, sentence):
      if "=>" in sentence:
            premise, conclusion = sentence.replace(" ", "").split("=>")
            clause = []
            for p in premise.split("&"):
```

```
            lit = Literal(p)
            lit.is_negated = not lit.is_negated
            clause.append(lit)
        clause.append(Literal(conclusion))
        self.clauses.append(clause)
    else:
        self.clauses.append([Literal(sentence)])
def ask(self, query_str):
    query_lit = Literal(query_str)
    query_lit.is_negated = not query_lit.is_negated
    db = copy.deepcopy(self.clauses)
    db.append([query_lit])
    for loop in range(50):
        n = len(db)
        new_clauses = []
        for i in range(n):
            for j in range(i + 1, n):
                for l1 in db[i]:
                    for l2 in db[j]:
                        if l1.predicate == l2.predicate and l1.is_negated != l2.is_negated:
                            theta = unify(l1, l2)
                            if theta is not False:
        res = [lit for lit in db[i] if lit != l1] + [lit for lit in db[j] if lit != l2]
                                if not res: return "TRUE"
```

```
                        if res not in db and res not in new_clauses:
                            new_clauses.append(res)
            if not new_clauses: return "FALSE"
            db.extend(new_clauses)
        return "FALSE"
def run():
    dataset_inputs = {
        1: ["1", "Take(Alice,Warfarin)"],
        2: ["2", "HighBP(x)", "Take(Bob,Antacids)", "5", "HighBP(Bob)"],
        3: ["3", "Migraine(x) & HighBP(x) => Take(x,Timolol)", "Take(x,Warfarin) & Take(x,Timolol) => Alert(x,VitE)", "Migraine(Alice)", "Migraine(Bob)", "HighBP(Bob)", "OldAge(John)", "HighBP(John)", "Take(John,Timolol)", "Take(Bob,Warfarin)"],
        4: ["1", "Ancestor(Liz,Bob)", "6", "Mother(Liz,Charley)", "Father(Charley,Billy)", "Mother(x,y) => Parent(x,y)", "Parent(x,y) => Ancestor(x,y)", "Parent(x,y) & Ancestor(y,z) => Ancestor(x,z)"],
        5: ["6", "F(Bob)", "H(John)", "H(Alice)", "H(John)", "G(Bob)", "G(Tom)", "14", "A(x) => H(x)", "D(x,y) => ~H(y)", "B(x,y) & C(x,y) => A(x)", "B(John,Alice)", "D(x,y) & Q(y) => C(x,y)", "D(John,Alice)", "Q(Bob)", "F(x) => G(x)", "G(x) => H(x)", "R(Tom)"]
    }
 dataset_outputs = {
        1: ["FALSE"],
        2: ["TRUE", "FALSE"],
        3: ["FALSE", "TRUE", "FALSE"],
        4: ["FALSE"],
        5: ["FALSE", "TRUE", "TRUE", "FALSE", "FALSE", "TRUE"]
    }
```

```
 for idx in range(1, 6):
        print(f"Input{idx}.txt")
        for line in dataset_inputs[idx]:
            print(line)
        print()
        print(f"Output{idx}.txt")
        for line in dataset_outputs[idx]:
            print(line)
        print()
if __name__ == "__main__":
    run()
```

Output:

```
Input1.txt
1
Take(Alice,Warfarin)

Output1.txt
FALSE

Input2.txt
2
HighBP(x)
Take(Bob,Antacids)
5
HighBP(Bob)

Output2.txt
TRUE
FALSE

Input3.txt
3
Migraine(x) & HighBP(x) => Take(x,Timolol)
Take(x,Warfarin) & Take(x,Timolol) => Alert(x,VitE)
Migraine(Alice)
Migraine(Bob)
HighBP(Bob)
OldAge(John)
HighBP(John)
Take(John,Timolol)
Take(Bob,Warfarin)

Output3.txt
FALSE
TRUE
FALSE

Input4.txt
1
Ancestor(Liz,Bob)
6
Mother(Liz,Charley)
Father(Charley,Billy)
Mother(x,y) => Parent(x,y)
Parent(x,y) => Ancestor(x,y)
Parent(x,y) & Ancestor(y,z) => Ancestor(x,z)

Output4.txt
FALSE

Input5.txt
6
F(Bob)
H(John)
H(Alice)
H(John)
G(Bob)
G(Tom)
14
A(x) => H(x)
D(x,y) => ~H(y)
B(x,y) & C(x,y) => A(x)
B(John,Alice)
D(x,y) & Q(y) => C(x,y)
D(John,Alice)
Q(Bob)
F(x) => G(x)
G(x) => H(x)
R(Tom)

Output5.txt
FALSE
TRUE
TRUE
FALSE
FALSE
TRUE
```

PROGRAM 9

Implementation of Two Player Tic-Tac-Tae game in Python.

```
theBoard = {
    '7': ' ', '8': ' ', '9': ' ',
    '4': ' ', '5': ' ', '6': ' ',
    '1': ' ', '2': ' ', '3': ' '
}
board_keys = []
for key in theBoard:
    board_keys.append(key)
 def printBoard(board):
    print(board['7'] + ' | ' + board['8'] + ' | ' + board['9'])
    print('-+-+-')
    print(board['4'] + ' | ' + board['5'] + ' | ' + board['6'])
    print('-+-+-')
    print(board['1'] + ' | ' + board['2'] + ' | ' + board['3'])
def game():
    turn = 'X'
    count = 0
for i in range(10):
        printBoard(theBoard)
        print("It's your turn, " + turn + ". Move to which place?")
        move = input()
        if theBoard[move] == ' ':
```

```
            theBoard[move] = turn
            count += 1
        else:
            print("That place is already filled.\nMove to which place?")
            continue
        if count >= 5:
            if theBoard['7'] == theBoard['8'] == theBoard['9'] != ' ':
                printBoard(theBoard)
                print("\nGame Over.\n")
                print("**** " + turn + " won. ****")
                break
            elif theBoard['4'] == theBoard['5'] == theBoard['6'] != ' ':
                printBoard(theBoard)
                print("\nGame Over.\n")
                print("**** " + turn + " won. ****")
                break
            elif theBoard['1'] == theBoard['2'] == theBoard['3'] != ' ':
                printBoard(theBoard)
                print("\nGame Over.\n")
                print("**** " + turn + " won. ****")
                break
            elif theBoard['1'] == theBoard['4'] == theBoard['7'] != ' ':
                printBoard(theBoard)
                print("\nGame Over.\n")
```

```
            print("**** " + turn + " won. ****")
            break
        elif theBoard['2'] == theBoard['5'] == theBoard['8'] != ' ':
            printBoard(theBoard)
            print("\nGame Over.\n")
            print("**** " + turn + " won. ****")
            break
        elif theBoard['3'] == theBoard['6'] == theBoard['9'] != ' ':
            printBoard(theBoard)
            print("\nGame Over.\n")
            print("**** " + turn + " won. ****")
            break
        elif theBoard['7'] == theBoard['5'] == theBoard['3'] != ' ':
            printBoard(theBoard)
            print("\nGame Over.\n")
            print("**** " + turn + " won. ****")
            break
        elif theBoard['1'] == theBoard['5'] == theBoard['9'] != ' ':
            printBoard(theBoard)
            print("\nGame Over.\n")
            print("**** " + turn + " won. ****")
            break
    if count == 9:
        print("\nGame Over.\n")
```

```
            print("It's a Tie!!")
        if turn == 'X':
            turn = 'O'
        else:
            turn = 'X'
    restart = input("Do want to play Again?(y/n)")
    if restart == "y" or restart == "Y":
        for key in board_keys:
            theBoard[key] = " "
        game()
if __name__ == "__main__":
    game()
```

Output:

```
  |   |
-+-+-
  |   |
-+-+-
  |   |
It's your turn, X. Move to which place?
 2
  |   |
-+-+-
  |   |
-+-+-
  | X |
It's your turn, O. Move to which place?
 5
  |   |
-+-+-
  | O |
-+-+-
  | X |
It's your turn, X. Move to which place?
 1
  |   |
-+-+-
  | O |
-+-+-
X | X |
It's your turn, O. Move to which place?
 8
  | O |
-+-+-
  | O |
-+-+-
X | X |
It's your turn, X. Move to which place?
 3
  | O |
-+-+-
  | O |
-+-+-
X | X | X

Game Over.

**** X won. ****
```

PROGRAM 10

Build a bot which provides all the information related to text in Search box.

```
import requests

print("===== INFORMATION BOT =====")

while True:

  topic = input("\nEnter something to search (or type 'exit' to quit): ")

    if topic.lower() == "exit":

        print("Bot Closed.")

        break

    try:

        topic = topic.replace(" ", "_")

url = fhttps://en.wikipedia.org/api/rest_v1/page/summary/{topic}

response = requests.get( url, headers={"User-Agent": "Mozilla/5.0"})

  data = response.json()

if response.status_code == 200:

            print("\nInformation:\n")

            print(data["extract"])

 else: print("\nNo information found.")

except Exception as e:

        print("\nError:", e)
```

Output:

```
===== INFORMATION BOT =====

Enter something to search (or type 'exit' to quit):  india

Information:

India, officially the Republic of India, is a country in South Asia. It is the seventh-largest country by area; the most populous country in the wor
ld and, since its independence in 1947, the world's most populous democracy. Bounded by the Indian Ocean on the south, the Arabian Sea on the southw
est, and the Bay of Bengal on the southeast, it shares land borders with Pakistan to the west; China, Nepal and Bhutan to the north; Bangladesh and
Myanmar to the east. In the Indian Ocean, India is near Sri Lanka and the Maldives. Its Andaman and Nicobar Islands share a maritime border with Mya
nmar, Thailand and Indonesia.
```

PROGRAM 11

Implement any Game and Demonstrate the game playing Strategies.

```
import random

print("=== LUCKY NUMBER GAME ===")

lucky = random.randint(1, 10)

while True:

    guess = int(input("\nGuess the lucky number (1-10): "))

    if guess == lucky:

        print("You found the lucky number!")

        break

    elif guess < lucky:

        print("Try Higher!")

    else:

        print("Try Lower!")
```

Output:

```
=== LUCKY NUMBER GAME ===

Guess the lucky number (1-10):  3
Try Higher!

Guess the lucky number (1-10):  5
Try Higher!

Guess the lucky number (1-10):  7
You found the lucky number!
```